Lecture 03: Linking Relocatable ELF File

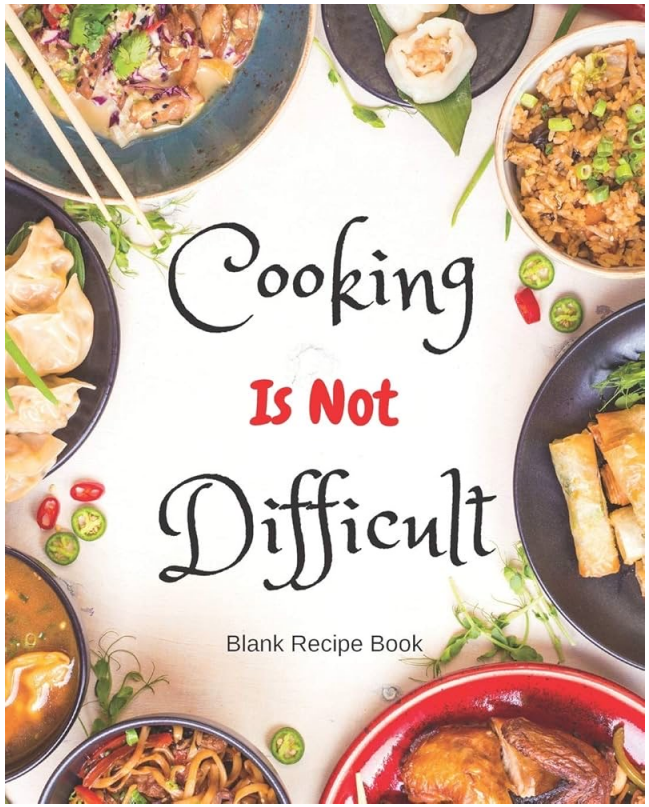
Vivek Kumar

Computer Science and Engineering

IIIT Delhi

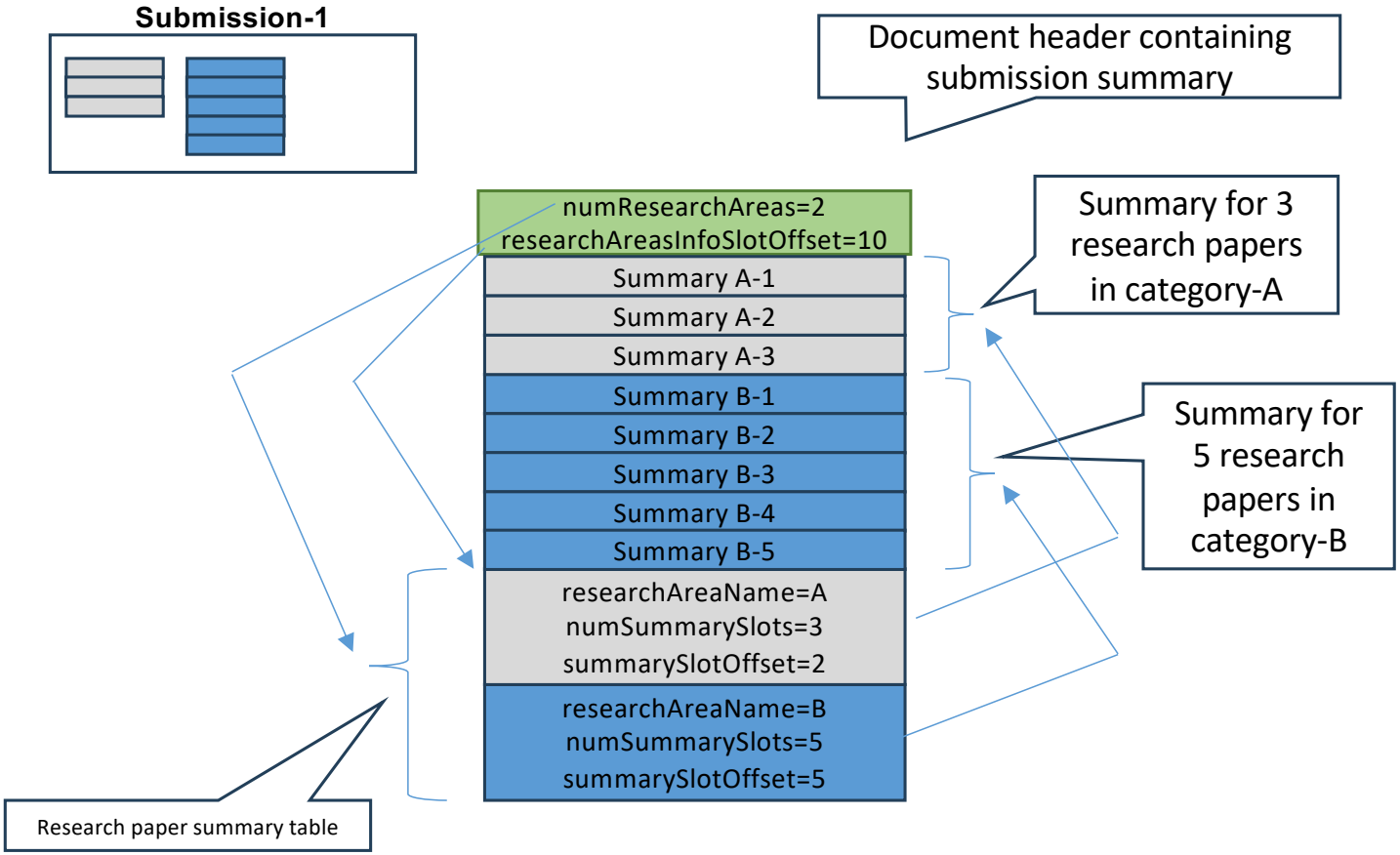
vivekk@iiitd.ac.in

Last Lecture: The Recipe book



- Who is the cook?
  - The one who executes the service
    - E.g., the Operating System!
- What is the recipe?
  - A well-structured format for recording recipe that the cook can understand
    - E.g., the executable file format
- The index page in the book
  - Header in the executable file format that helps in browsing the book
- The ingredients in the recipe?
  - Program data stored in the executable file format
- The red sauce
  - Statically linked reference if you download its recipe and permanently attach to your recipe book
  - Dynamically linked reference if you look it up online only while actually cooking
- Who prepares the recipe book??
  - Its you who is playing the role of a compiler!

Last Lecture: Sample Assignment



- I proposed a specific layout to overcome this challenge, and to make it easy to understand student's document
- I can easily "**load**" the student "**file**" from any number of students in my "**memory**" if they follow a standard "**format**" to store information, e.g., as the one shown below for the **Submission-1** shown here

The Story So Far...

- A **cook** *(OS)* manages a **kitchen** *(computer)*
  - Lecture 01
- The job of a cook is to do **cooking** *(execute binaries)* by reading **recipes** *(executable file)* in a **standard format** *(ELF)*
  - Lecture 02
- The cook has received recipe of a dish in **different files** *(relocatable files)*. It will have to use a **stapler** *(linker)* to stitch these files to make a final recipe
  - Today's point of discussion!

Today's Class

- Linking relocatable ELF files
  - Symbol resolution
- Revisiting Executable File Format (ELF)

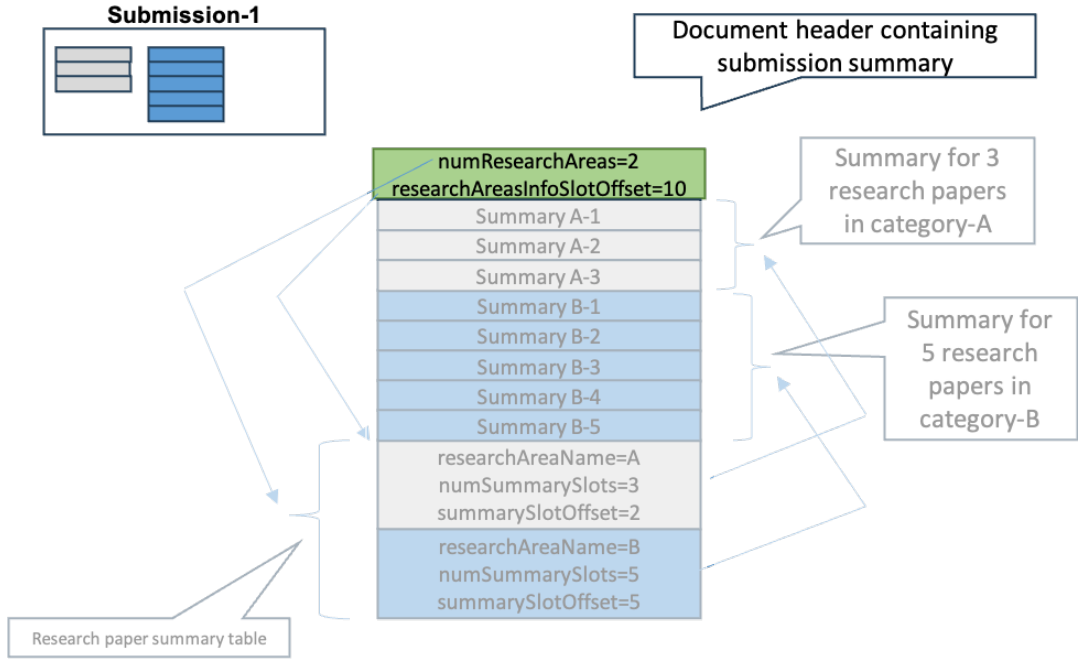
ELF Header (The index page in Book)

```
#define EI_NIDENT 16

typedef struct {
        unsigned char   e_ident[EI_NIDENT];
        Elf32_Half      e_type;
        Elf32_Half      e_machine;
        Elf32_Word      e_version;
        Elf32_Addr      e_entry;
        Elf32_Off       e_phoff;
        Elf32_Off       e_shoff;
        Elf32_Word      e_flags;
        Elf32_Half      e_ehsize;
        Elf32_Half      e_phentsize;
        Elf32_Half      e_phnum;
        Elf32_Half      e_shentsize;
        Elf32_Half      e_shnum;
        Elf32_Half      e_shstrndx;
} Elf32_Ehdr;
```

```
vivek@possum:~/os23$ readelf -h ./a.out
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1160
  Start of program headers:          64 (bytes into file)
  Start of section headers:          14320 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

Image source: https://refspecs.linuxfoundation.org/elf/gabi4+/ch4.eheader.html

```
typedef struct {
    short numResearchAreas;              // All three summarized?
    short researchAreasInfoSlotOffset;   // Column index
} CSE231_Ehdr;
```



ELF Header

```
#define EI_NIDENT 16

typedef struct {
        unsigned char   e_ident[EI_NIDENT];
        Elf32_Half      e_type;
        Elf32_Half      e_machine;
        Elf32_Word      e_version;
        Elf32_Addr      e_entry;
        Elf32_Off       e_phoff;
        Elf32_Off       e_shoff;
        Elf32_Word      e_flags;
        Elf32_Half      e_ehsize;
        Elf32_Half      e_phentsize;
        Elf32_Half      e_phnum;
        Elf32_Half      e_shentsize;
        Elf32_Half      e_shnum;
        Elf32_Half      e_shstrndx;
} Elf32_Ehdr;
```

Identification for object file

Object file type

Entry point address for starting the executable

Program Header Table (PHT) file offset

Section Header Table (SHT) file offset

ELF header size in bytes (fixed)

Size in bytes of each entry in PHT

Total number of entries in PHT

Size in bytes of each entry in SHT

Total number of entries in SHT

Index of the section called as "String Table"

Image source: https://refspecs.linuxfoundation.org/elf/gabi4+/ch4.eheader.html

Entry Point?

```
vivek@possum:~/elf_os-12$ readelf -h ./a.out
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                           Intel 80386
  Version:                           0x1
  Entry point address:               0x3e0
  Start of program headers:          52 (bytes into file)
  Start of section headers:          6060 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         9
  Size of section headers:           40 (bytes)
  Number of section headers:         29
  Section header string table index: 28
```

Entry point basically provides the location of the first instruction that has to be executed (?)

Entry Point?

```
vivek@possum:~/elf_os-12$ readelf -h ./a.out
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF32
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Shared object file)
  Machine:                           Intel 80386
  Version:                           0x1
  Entry point address:               0x3e0
  Start of program headers:          52 (bytes into file)
  Start of section headers:          6060 (bytes into file)
  Flags:                             0x0
  Size of this header:               52 (bytes)
  Size of program headers:           32 (bytes)
  Number of program headers:         9
  Size of section headers:           40 (bytes)
  Number of section headers:         29
  Section header string table index: 28
vivek@possum:~/elf_os-12$ objdump -d ./a.out | grep -A16 3e0
000003e0 <_start>:
 3e0:   31 ed                   xor    %ebp,%ebp
 3e2:   5e                      pop    %esi
 3e3:   89 e1                   mov    %esp,%ecx
 3e5:   83 e4 f0                and    $0xfffffff0,%esp
 3e8:   50                      push   %eax
 3e9:   54                      push   %esp
 3ea:   52                      push   %edx
 3eb:   e8 22 00 00 00          call   412 <_start+0x32>
 3f0:   81 c3 e8 1b 00 00       add    $0x1be8,%ebx
 3f6:   8d 83 f8 e5 ff ff       lea    -0x1a08(%ebx),%eax
 3fc:   50                      push   %eax
 3fd:   8d 83 98 e5 ff ff       lea    -0x1a68(%ebx),%eax
 403:   50                      push   %eax
 404:   51                      push   %ecx
 405:   56                      push   %esi
 406:   ff b3 20 00 00 00       pushl  0x20(%ebx)
 40c:   e8 af ff ff ff          call   3c0 <__libc_start_main@plt>
```

Entry point address

Entry point actual method

Entry point address

_start calls __libc_start_main with the address of user main as a parameter (invoked from here)

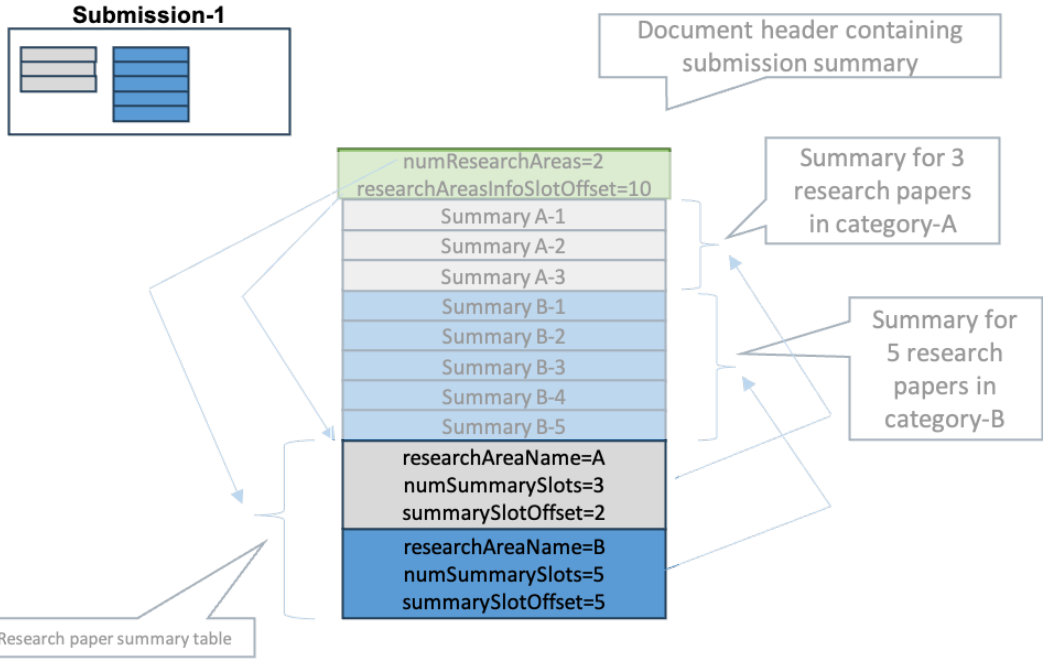
ELF Section Header (Chapter Details)

```
typedef struct {
    Elf32_Word      sh_name;
    Elf32_Word      sh_type;
    Elf32_Word      sh_flags;
    Elf32_Addr      sh_addr;
    Elf32_Off       sh_offset;
    Elf32_Word      sh_size;
    Elf32_Word      sh_link;
    Elf32_Word      sh_info;
    Elf32_Word      sh_addralign;
    Elf32_Word      sh_entsize;
} Elf32_Shdr;
```

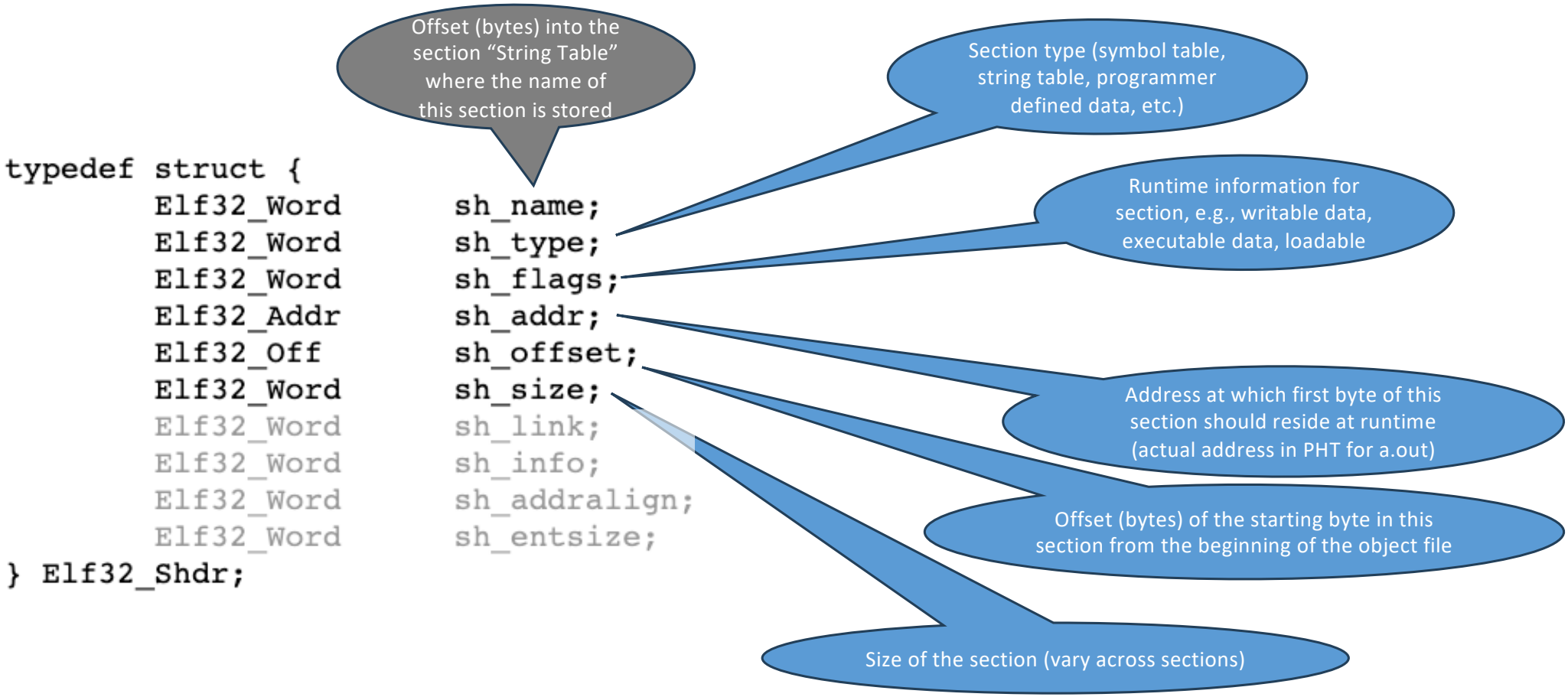
ELF_Shdr does not store name directly

```
typedef struct {
  char* researchAreaName;      // "A" or "B" or "C" ?
short numSummarySlots;         // Total papers summarized
short summarySlotOffset;       // Index to its first summary line
} CSE231_Shdr;
```

```
vivek@possum:~/elf_os-12$ readelf -S hello.o
There are 17 section headers, starting at offset 0x3d4:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .group            GROUP           00000000 000034 000008 04     14  16  4
  [ 2] .text             PROGBITS        00000000 00003c 000059 00  AX  0   0  1
  [ 3] .rel.text         REL             00000000 0002e4 000040 08   I 14   2  4
  [ 4] .data             PROGBITS        00000000 000098 000004 00  WA  0   0  4
  [ 5] .bss              NOBITS          00000000 00009c 000000 00  WA  0   0  1
  [ 6] .rodata           PROGBITS        00000000 00009c 000025 00   A  0   0  1
  [ 7] .data.rel.local   PROGBITS        00000000 0000c4 000004 00  WA  0   0  4
  [ 8] .rel.data.rel.loc REL             00000000 000324 000008 08   I 14   7  4
  [ 9] .text.__x86.get_p PROGBITS        00000000 0000c8 000004 00 AXG  0   0  1
  [10] .comment          PROGBITS        00000000 0000cc 00002a 01  MS  0   0  1
  [11] .note.GNU-stack   PROGBITS        00000000 0000f6 000000 00      0   0  1
  [12] .eh_frame         PROGBITS        00000000 0000f8 000060 00   A  0   0  4
  [13] .rel.eh_frame     REL             00000000 00032c 000010 08   I 14  12  4
  [14] .symtab           SYMTAB          00000000 000158 000130 10     15  12  4
  [15] .strtab           STRTAB          00000000 000288 00005b 00      0   0  1
  [16] .shstrtab         STRTAB          00000000 00033c 000096 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  p (processor specific)
```



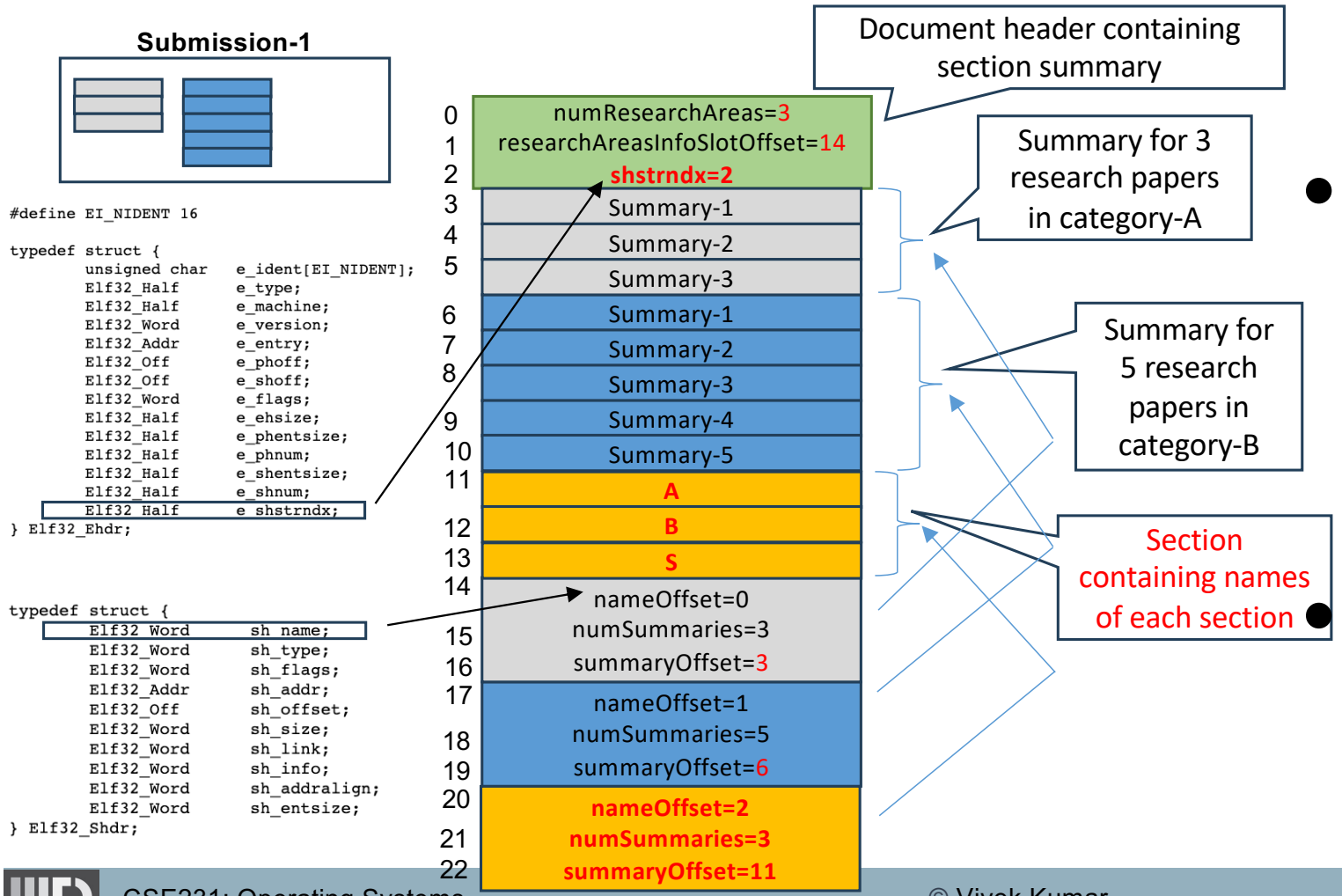
ELF Section Header



How Linker Reads Section Names?

- The section header does not contains the name of its corresponding section
  - It's structure has a member that contains an offset into the ELF file where the name of this section is stored
- The names of all the sections are stored inside **.shstrtab** section
- Each section header stores the offset into the .shstrtab section (not the SHDR but the actual section) where the name of that section is stored as a null terminated string

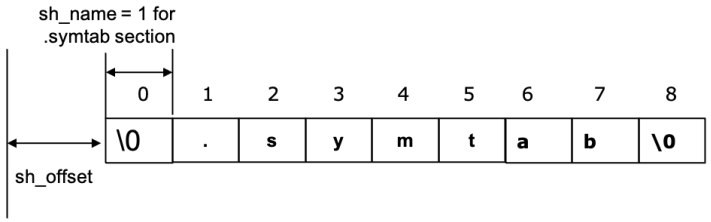
Motivating Example



- It is the same motivational example we discussed in Lecture 02
  - Changes shown in **orange** and **red** color
- We added a new section "S" that contains the name of all the sections in this document
  - Document header contains the index of this particular section summary
  - Name of this section ("S") will also be now stored in the actual content of this section
- Each section summary now also contains **offset from the start of the content of section "S"**, where the name is stored

The ".shstrtab" Section in ELF

```
vivek@possum:~/elf_os-12$ readelf -p .shstrtab hello

String dump of section '.shstrtab':
  [     1]  .symtab
  [     9]  .strtab
  [    11]  .shstrtab
  [    1b]  .interp
  [    23]  .note.ABI-tag
  [    31]  .note.gnu.build-id
  [    44]  .gnu.hash
  [    4e]  .dynsym
  [    56]  .dynstr
  [    5e]  .gnu.version
  [    6b]  .gnu.version_r
  [    7a]  .rel.dyn
  [    83]  .rel.plt
  [    8c]  .init
  [    92]  .plt.got
  [    9b]  .text
  [    a1]  .fini
  [    a7]  .rodata
  [    af]  .eh_frame_hdr
  [    bd]  .eh_frame
  [    c7]  .init_array
  [    d3]  .fini_array
  [    df]  .dynamic
  [    e8]  .data
  [    ee]  .bss
  [    f3]  .comment
```



- It is a String Table Section
- Stores the name of each sections in the binary
  - String (NULL terminated)
- As with other sections, this section is also a contiguous chunk of memory

First column is the offset (hex value) from the start for the first character of the corresponding section name. First entry starts at index 1

Few Other Interesting Sections

```
vivek@possum:~/elf_os-12$ readelf -S hello
There are 29 section headers, starting at offset 0x17fc:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS        00000154 000154 000013 00   A  0   0  1
  [ 2] .note.ABI-tag     NOTE            00000168 000168 000020 00   A  0   0  4
  [ 3] .note.gnu.build-i NOTE            00000188 000188 000024 00   A  0   0  4
  [ 4] .gnu.hash         GNU_HASH        000001ac 0001ac 000020 04   A  5   0  4
  [ 5] .dynsym           DYNSYM          000001cc 0001cc 000080 10   A  6   1  4
  [ 6] .dynstr           STRTAB          0000024c 00024c 00009d 00   A  0   0  1
  [ 7] .gnu.version      VERSYM          000002ea 0002ea 000010 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED         000002fc 0002fc 000030 00   A  6   1  4
  [ 9] .rel.dyn          REL             0000032c 00032c 000048 08   A  5   0  4
  [10] .rel.plt          REL             00000374 000374 000010 08  AI  5  22  4
  [11] .init             PROGBITS        00000384 000384 000023 00  AX  0   0  4
  [12] .plt              PROGBITS        000003b0 0003b0 000030 04  AX  0   0 16
  [13] .plt.got          PROGBITS        000003e0 0003e0 000010 08  AX  0   0  8
  [14] .text             PROGBITS        000003f0 0003f0 000202 00  AX  0   0 16
  [15] .fini             PROGBITS        000005f4 0005f4 000014 00  AX  0   0  4
  [16] .rodata           PROGBITS        00000608 000608 00002d 00   A  0   0  4
  [17] .eh_frame_hdr     PROGBITS        00000638 000638 00003c 00   A  0   0  4
  [18] .eh_frame         PROGBITS        00000674 000674 0000fc 00   A  0   0  4
  [19] .init_array       INIT_ARRAY      00001ed8 000ed8 000004 04  WA  0   0  4
  [20] .fini_array       FINI_ARRAY      00001edc 000edc 000004 04  WA  0   0  4
  [21] .dynamic          DYNAMIC         00001ee0 000ee0 0000f8 08  WA  6   0  4
  [22] .got              PROGBITS        00001fd8 000fd8 000028 04  WA  0   0  4
  [23] .data             PROGBITS        00002000 001000 000010 00  WA  0   0  4
  [24] .bss              NOBITS          00002020 001010 400020 00  WA  0   0 32
  [25] .comment          PROGBITS        00000000 001010 000029 01  MS  0   0  1
  [26] .symtab           SYMTAB          00000000 00103c 000460 10     27  43  4
  [27] .strtab           STRTAB          00000000 00149c 000264 00      0   0  1
  [28] .shstrtab         STRTAB          00000000 001700 0000fc 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  p (processor specific)
```

```
vivek@possum:~/elf_os-12$ cat hello.c
#include<stdio.h>

int my_global1 = 1, my_global2[1024*1024];
char* str = "Updated value of my_global1";

int main() {
  my_global1 += 1;
  printf("%s = %d\n", str, my_global1);
  return 0;
}
```

Executable Instructions (".text")

```
vivek@possum:~/elf_os-l2$ objdump -d hello.o | grep -A20 text
Disassembly of section .text:

00000000 <main>:
   0:   8d 4c 24 04             lea    0x4(%esp),%ecx
   4:   83 e4 f0                and    $0xfffffff0,%esp
   7:   ff 71 fc                pushl  -0x4(%ecx)
   a:   55                      push   %ebp
   b:   89 e5                   mov    %esp,%ebp
   d:   53                      push   %ebx
   e:   51                      push   %ecx
   f:   e8 fc ff ff ff          call   10 <main+0x10>
  14:   05 01 00 00 00          add    $0x1,%eax
  19:   8b 90 00 00 00 00       mov    0x0(%eax),%edx
  1f:   83 c2 01                add    $0x1,%edx
  22:   89 90 00 00 00 00       mov    %edx,0x0(%eax)
  28:   8b 88 00 00 00 00       mov    0x0(%eax),%ecx
  2e:   8b 90 00 00 00 00       mov    0x0(%eax),%edx
  34:   83 ec 04                sub    $0x4,%esp
  37:   51                      push   %ecx
  38:   52                      push   %edx
  39:   8d 90 1c 00 00 00       lea    0x1c(%eax),%edx
```

```
vivek@possum:~/elf_os-l2$ objdump -d hello | grep -A20  text
Disassembly of section .text:

000003f0 <_start>:
 3f0:   31 ed                   xor    %ebp,%ebp
 3f2:   5e                      pop    %esi
 3f3:   89 e1                   mov    %esp,%ecx
 3f5:   83 e4 f0                and    $0xfffffff0,%esp
 3f8:   50                      push   %eax
 3f9:   54                      push   %esp
 3fa:   52                      push   %edx
 3fb:   e8 22 00 00 00          call   422 <_start+0x32>
 400:   81 c3 d8 1b 00 00       add    $0x1bd8,%ebx
 406:   8d 83 18 e6 ff ff       lea    -0x19e8(%ebx),%eax
 40c:   50                      push   %eax
 40d:   8d 83 b8 e5 ff ff       lea    -0x1a48(%ebx),%eax
 413:   50                      push   %eax
 414:   51                      push   %ecx
 415:   56                      push   %esi
 416:   ff b3 20 00 00 00       pushl  0x20(%ebx)
 41c:   e8 af ff ff ff          call   3d0 <__libc_start_main@
 421:   f4                      hlt
```

- Stores bytes corresponding to program execution (user code and the GNU C library supporting code)
- The above output shows the content of ".text" section for relocatable v/s executable
  - *'A20' prints 20 lines after a match is found by grep command*

Section ".rodata"

```
vivek@possum:~/elf_os-12$ cat hello.c
#include<stdio.h>

int my_global1 = 1, my_global2[1024*1024];
char* str = "Updated value of my_global1";

int main() {
  my_global1 += 1;
  printf("%s = %d\n", str, my_global1);
  return 0;
}
```

```
vivek@possum:~/elf_os-12$ readelf -p .rodata hello

String dump of section '.rodata':
  [     8]  Updated value of my_global1
  [    24]  %s = %d^J
```

- Stores all the string literals defined in the program
  - First column in readelf output is the offset from the start for that particular string
- Question
  - Would the content change across relocatable and executable?

Section ".data"

```
vivek@possum:~/elf_os-12$ cat hello.c
#include<stdio.h>

int my_global1 = 1, my_global2[1024*1024];
char* str = "Updated value of my_global1";

int main() {
  my_global1 += 1;
  printf("%s = %d\n", str, my_global1);
  return 0;
}
```

```
vivek@possum:~/elf_os-12$ readelf -S hello.o | grep '\.data \| Name'
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 4] .data             PROGBITS        00000000 000098 000004 00  WA  0   0  4
```

- Contains the **initialized value** of all the global/static variables from the user code (not the variable name)
  - In our running example it is the **value** of the variable "my_global1"
    - Why not the value of "str"?
- "WA" flag indicates that the section content is to be loaded at runtime in memory and is writable
- PROGBITS indicating it contains information defined by the user code
- Where is the name of the global variable stored?

Sections ".symtab" and ".strtab"

```
vivek@possum:~/elf_os-l2$ readelf -p .symtab hello.o

String dump of section '.symtab':
  [    d4]
  [    da]  @
  [    f0]  #
  [    f8]  Y
  [   100]  (
  [   110]  >
  [   120]  T

vivek@possum:~/elf_os-l2$ readelf -p .strtab hello.o

String dump of section '.strtab':
  [     1]  hello.c
  [     9]  my_global1
  [    14]  my_global2
  [    1f]  str
  [    23]  main
  [    28]  __x86.get_pc_thunk.ax
  [    3e]  _GLOBAL_OFFSET_TABLE_
  [    54]  printf
```

- ".symtab" is like a directory to map the name of the symbols (present inside .strtab) with the actual content (value) of that symbol
  - E.g., for the symbol "my_global1", symbol table ".symtab" will contain an index into the string table ".strtab" where the name "my_global1" is stored

Section ".bss"

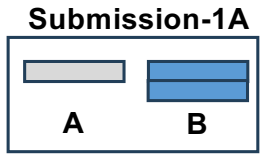
```
vivek@possum:~/elf_os-12$ cat hello.c
#include<stdio.h>

int my_global1 = 1, my_global2[1024*1024];
char* str = "Updated value of my_global1";

int main() {
  my_global1 += 1;
  printf("%s = %d\n", str, my_global1);
  return 0;
}
```

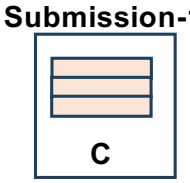
Hex

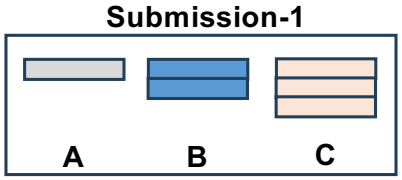
```
vivek@possum:~/elf_os-12$ readelf -S hello | grep '\.bss\|Name'
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [24] .bss              NOBITS          00002020 001010 400020 00  WA  0   0 32
```

- .bss (Block Started by Symbol) section indicates the total memory to be allocated at runtime for **uninitialized** global/static variables
  - In our running example it is the **value** of the variable "my_global2"
- "WA" flag indicates that the section content is to be loaded at runtime in memory and is writable
- NOBITS indicating it relates to the user code, but it doesn't occupy any space in the object file

How Linker Stitches Two Relocatable (.o) ?

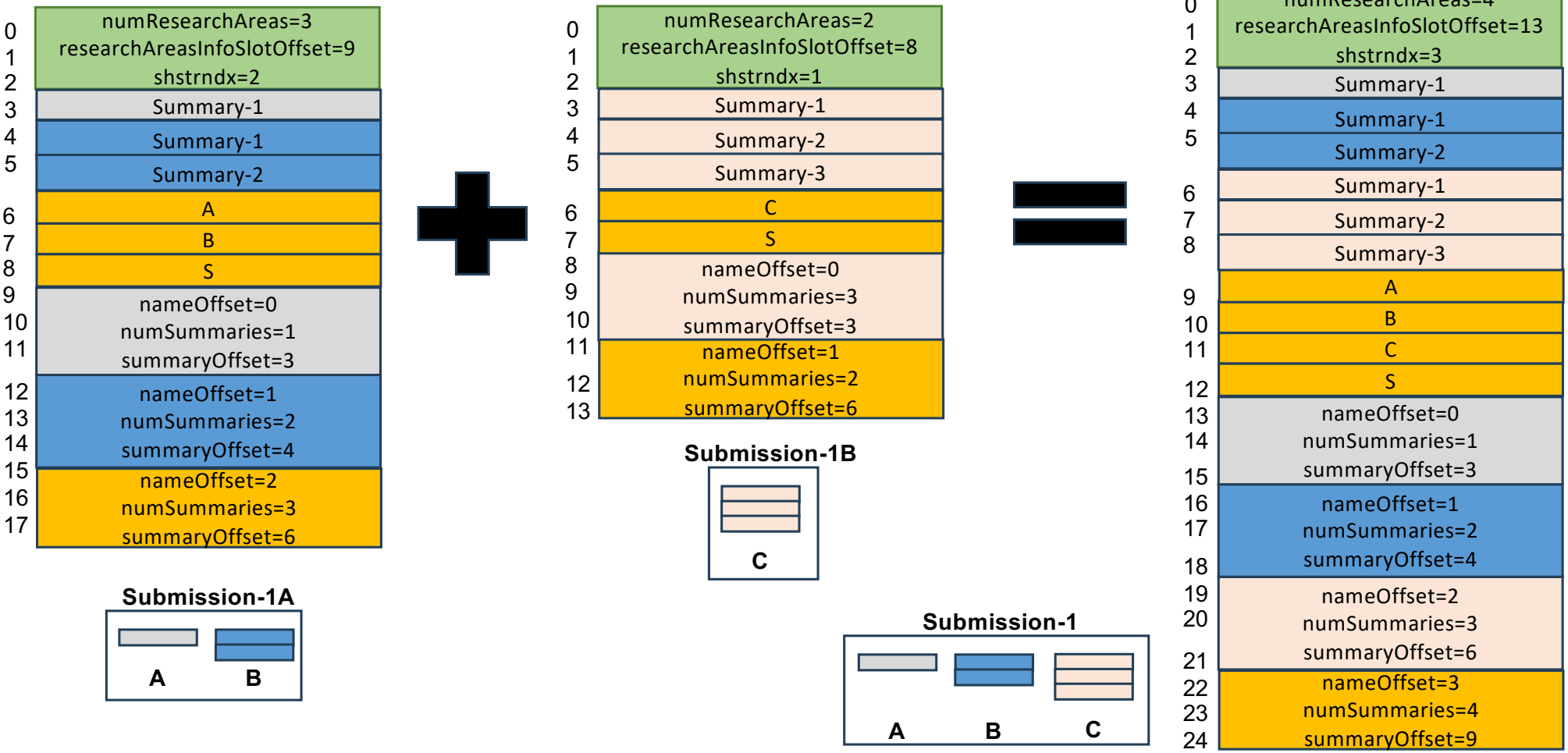




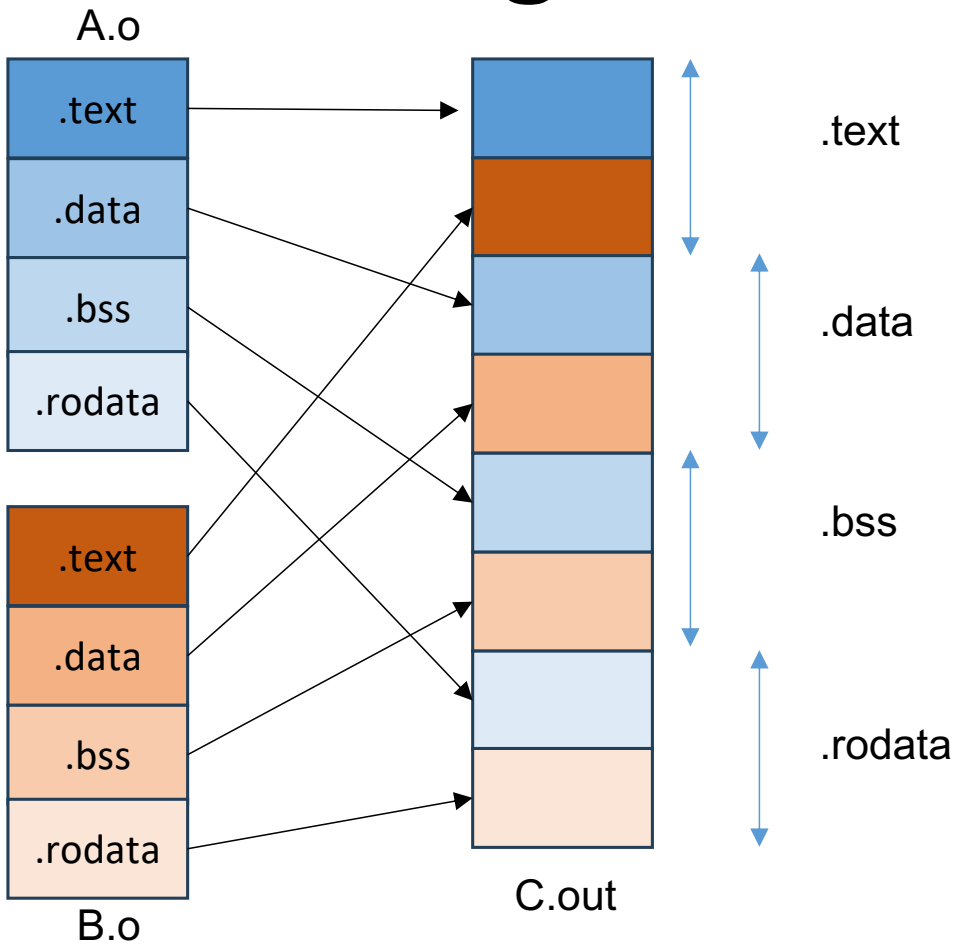


- Let us revisit our motivational example of student assignment to summarize the research areas in a single line
- I announced upfront that there must be a single submission from each student, but one student submitted two separate files BUT in correct format (our modified ELF)
  - Submission-1A having summaries for areas A and B
  - Submission-1B having summary for area C
- I asked the student to **STITCH** both the submissions into a single submission (the job of a linker!)
  - Submission-1 having summaries for A, B, and C

How Linker Stitches Two Relocatable (.o) ?

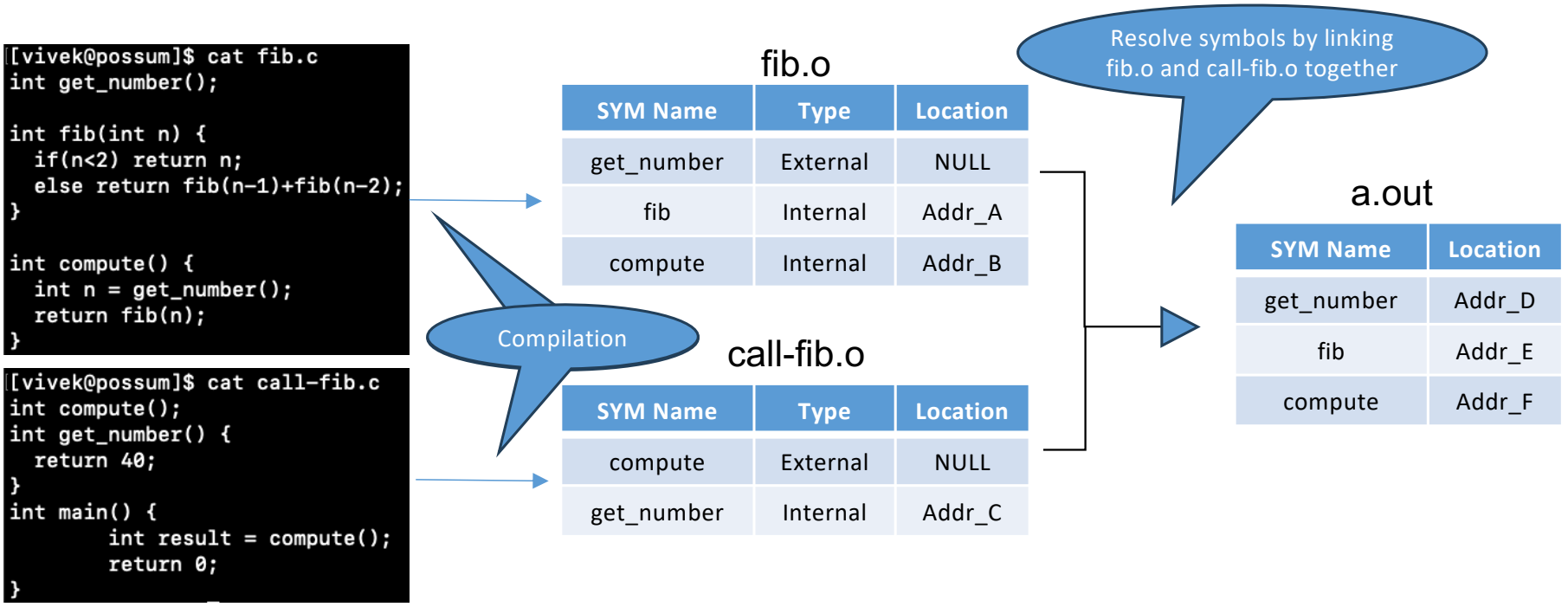


Linker: Merges the Sections



- **Relocation** is the process of merging the sections and resolving the symbol addresses

Linker: Relocation



Sections at Linking and Executable

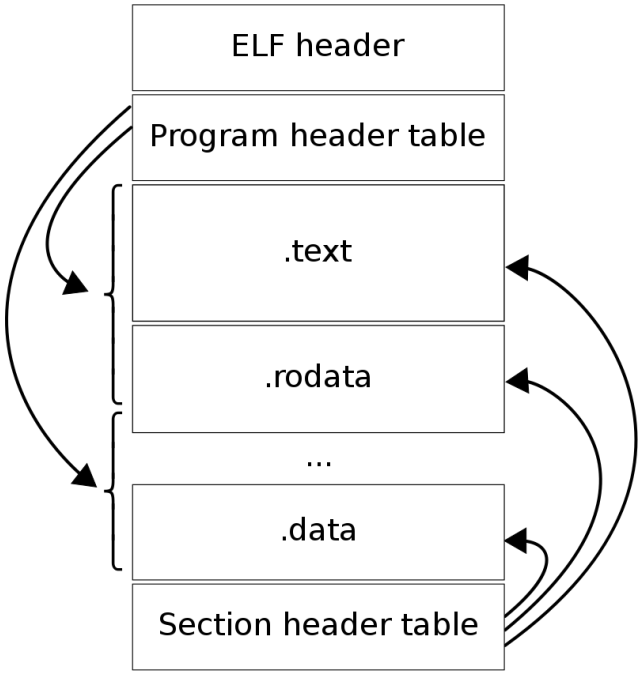
```
vivek@possum:~/elf_os-12$ readelf -S hello.o
There are 17 section headers, starting at offset 0x3d4:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .group            GROUP           00000000 000034 000008 04     14  16  4
  [ 2] .text             PROGBITS        00000000 00003c 000059 00  AX  0   0  1
  [ 3] .rel.text         REL             00000000 0002e4 000040 08   I 14   2  4
  [ 4] .data             PROGBITS        00000000 000098 000004 00  WA  0   0  4
  [ 5] .bss              NOBITS          00000000 00009c 000000 00  WA  0   0  1
  [ 6] .rodata           PROGBITS        00000000 00009c 000025 00   A  0   0  1
  [ 7] .data.rel.local   PROGBITS        00000000 0000c4 000004 00  WA  0   0  4
  [ 8] .rel.data.rel.loc REL             00000000 000324 000008 08   I 14   7  4
  [ 9] .text.__x86.get_p PROGBITS        00000000 0000c8 000004 00 AXG  0   0  1
  [10] .comment          PROGBITS        00000000 0000cc 00002a 01  MS  0   0  1
  [11] .note.GNU-stack   PROGBITS        00000000 0000f6 000000 00      0   0  1
  [12] .eh_frame         PROGBITS        00000000 0000f8 000060 00   A  0   0  4
  [13] .rel.eh_frame     REL             00000000 00032c 000010 08   I 14  12  4
  [14] .symtab           SYMTAB          00000000 000158 000130 10     15  12  4
  [15] .strtab           STRTAB          00000000 000288 00005b 00      0   0  1
  [16] .shstrtab         STRTAB          00000000 00033c 000096 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  p (processor specific)
```

Q: Why there are more number of sections in executable v/s relocatable file?

A: To assist the dynamic loader, the linker adds several more sections in the executable than in the relocatable

```
vivek@possum:~/elf_os-12$ readelf -S hello
There are 29 section headers, starting at offset 0x17fc:

Section Headers:
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 0]                   NULL            00000000 000000 000000 00      0   0  0
  [ 1] .interp           PROGBITS        00000154 000154 000013 00   A  0   0  1
  [ 2] .note.ABI-tag     NOTE            00000168 000168 000020 00   A  0   0  4
  [ 3] .note.gnu.build-i NOTE            00000188 000188 000024 00   A  0   0  4
  [ 4] .gnu.hash         GNU_HASH        000001ac 0001ac 000020 04   A  5   0  4
  [ 5] .dynsym           DYNSYM          000001cc 0001cc 000080 10   A  6   1  4
  [ 6] .dynstr           STRTAB          0000024c 00024c 00009d 00   A  0   0  1
  [ 7] .gnu.version      VERSYM          000002ea 0002ea 000010 02   A  5   0  2
  [ 8] .gnu.version_r    VERNEED         000002fc 0002fc 000030 00   A  6   1  4
  [ 9] .rel.dyn          REL             0000032c 00032c 000048 08   A  5   0  4
  [10] .rel.plt          REL             00000374 000374 000010 08  AI  5  22  4
  [11] .init             PROGBITS        00000384 000384 000023 00  AX  0   0  4
  [12] .plt              PROGBITS        000003b0 0003b0 000030 04  AX  0   0 16
  [13] .plt.got          PROGBITS        000003e0 0003e0 000010 08  AX  0   0  8
  [14] .text             PROGBITS        000003f0 0003f0 000202 00  AX  0   0 16
  [15] .fini             PROGBITS        000005f4 0005f4 000014 00  AX  0   0  4
  [16] .rodata           PROGBITS        00000608 000608 00002d 00   A  0   0  4
  [17] .eh_frame_hdr     PROGBITS        00000638 000638 00003c 00   A  0   0  4
  [18] .eh_frame         PROGBITS        00000674 000674 0000fc 00   A  0   0  4
  [19] .init_array       INIT_ARRAY      00001ed8 000ed8 000004 04  WA  0   0  4
  [20] .fini_array       FINI_ARRAY      00001edc 000edc 000004 04  WA  0   0  4
  [21] .dynamic          DYNAMIC         00001ee0 000ee0 0000f8 08  WA  6   0  4
  [22] .got              PROGBITS        00001fd8 000fd8 000028 04  WA  0   0  4
  [23] .data             PROGBITS        00002000 001000 000010 00  WA  0   0  4
  [24] .bss              NOBITS          00002020 001010 400020 00  WA  0   0 32
  [25] .comment          PROGBITS        00000000 001010 000029 01  MS  0   0  1
  [26] .symtab           SYMTAB          00000000 00103c 000460 10     27  43  4
  [27] .strtab           STRTAB          00000000 00149c 000264 00      0   0  1
  [28] .shstrtab         STRTAB          00000000 001700 0000fc 00      0   0  1
Key to Flags:
  W (write), A (alloc), X (execute), M (merge), S (strings), I (info),
  L (link order), O (extra OS processing required), G (group), T (TLS),
  C (compressed), x (unknown), o (OS specific), E (exclude),
  p (processor specific)
```

Executable and Linkable Format (ELF)



Note: For simplicity, we will only discuss ELF format for 32-bit object files in CSE231

Image source: https://en.wikipedia.org/wiki/Executable_and_Linkable_Format

- ELF header
  - Provides a roadmap for the entire file organization
    - Always at offset zero of the object file
    - Provides entry point address for execution
- Two views of an ELF file
  - Linkable view (relocatable file)
  - Execution view (executable file, shared object)
- Linkable view
  - Section header table
    - One section header for each section
  - Sections
    - Contains data required for linking
    - Machine code, global variables, (initialized and un-initialized), symbol tables, line mapping between machine code and original C code, etc.
- Execution (or Loader) view
  - Program header table
    - One program header for each segment
  - Segments
    - Created by merging several sections
    - Contains information required for by the loader for execution
- Contiguous chunk of memory (ELF header, PHT, SHT, each section, each segment)

Segments in Executable File

```
vivek@possum:~/elf_os-12$ readelf -l hello

Elf file type is DYN (Shared object file)
Entry point 0x3f0
There are 9 program headers, starting at offset 52

Program Headers:
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  PHDR           0x000034 0x00000034 0x00000034 0x00120 0x00120 R   0x4
  INTERP         0x000154 0x00000154 0x00000154 0x00013 0x00013 R   0x1
      [Requesting program interpreter: /lib/ld-linux.so.2]
  LOAD           0x000000 0x00000000 0x00000000 0x00770 0x00770 R E 0x1000
  LOAD           0x000ed8 0x00001ed8 0x00001ed8 0x00138 0x400168 RW  0x1000
  DYNAMIC        0x000ee0 0x00001ee0 0x00001ee0 0x000f8 0x000f8 RW  0x4
  NOTE           0x000168 0x00000168 0x00000168 0x00044 0x00044 R   0x4
  GNU_EH_FRAME   0x000638 0x00000638 0x00000638 0x0003c 0x0003c R   0x4
  GNU_STACK      0x000000 0x00000000 0x00000000 0x00000 0x00000 RW  0x10
  GNU_RELRO      0x000ed8 0x00001ed8 0x00001ed8 0x00128 0x00128 R   0x1

 Section to Segment mapping:
  Segment Sections...
   00
   01     .interp
   02     .interp .note.ABI-tag .note.gnu.build-id .gnu.hash .dynsym .dynstr .
gnu.version .gnu.version_r .rel.dyn .rel.plt .init .plt .plt.got .text .fini .
rodata .eh_frame_hdr .eh_frame
   03     .init_array .fini_array .dynamic .got .data .bss
   04     .dynamic
   05     .note.ABI-tag .note.gnu.build-id
   06     .eh_frame_hdr
   07
```

- Segments are created by merging several sections and each segment contains a specific type of information required for by the loader for execution (i.e., all the sections inside a segment are related)
  - Note the RE Flag for PHDR index 2 v/s RW Flag for index 3
- Not available in relocatable file

Summary: Portability with ELF

- Recall, one of the challenges of OS is to support portability
- Machine code is architecture dependent
- Having an architecture independent layout of object files helps in achieving portability
  - The OS will have same set of steps to load executables across different processor architectures (portability)
    - Easier to change and test the tools used for manipulating object files

Reference Materials

- Executable and Linkable Format
  - https://man7.org/linux/man-pages/man5/elf.5.html
  - https://www.sco.com/developers/devspecs/gabi41.pdf
  - https://wiki.osdev.org/ELF_Tutorial

Next Lecture

- Loading ELF Executable & procedure calling convention